SOFTWARE LIFECYCLE MANAGEMENT

A Microsoft Power Platform Solution for Software Procurement, Approvals, License Allocation and Renewal Management

DETAILED PROJECT REPORT

Arpit Kumar and Ankita Rani

Power Apps • Dataverse • Power Automate • Model-driven Apps

Power Platform Capstone Project • Project status: in progress

Document purpose and status

This report documents the Software Lifecycle Manager capstone: the business problem, project rationale, scope, architecture, data model, components already built, automation implemented, remaining work, testing strategy and final demonstration plan. It is intended both as a project record and as a practical build checklist.

Status discipline: this report distinguishes work explicitly confirmed in the project conversation from work proposed or still to be completed. A step described as planned must not be presented as completed until it has been implemented and tested in the training environment.

Status category	Meaning
Confirmed built / tested	The user explicitly reported successful creation or a successful test.
Built, but final verification needed	The component was configured, but its complete behaviour or security has not been demonstrated.
Planned / remaining	Included in the target solution but not yet confirmed as implemented.
Optional enhancement	Useful if time permits; not required for a credible capstone.

1. Executive summary

Software Lifecycle Manager (SLM) is an internal business application for managing the lifecycle of organizational software: discovering available products, requesting new software, obtaining approval, recording license allocation and monitoring renewal dates. The solution uses Dataverse as its central data store, a Canvas App as the employee-facing portal, a model-driven app as the procurement/IT workspace, and Power Automate for approval and renewal workflows.

The project was selected to be practical and recognizable without being a routine expense tracker or a generic CRUD application. Software subscriptions are recurring organizational commitments. Poor visibility can lead to duplicate purchases, unused licenses, missed renewals and fragmented approval records. SLM addresses these connected problems with a manageable, demonstrable workflow.

The Canvas App submission path has been tested successfully: a user selected software, submitted a request, and the resulting Dataverse record appeared with the expected fields and ReqStatus = Submitted. The model-driven app has been created with the four core tables and custom request views. Flow 1, the request approval engine, was reported as successfully tested. Flow 2, the license renewal monitor, has been started but its complete date logic, reminder actions and duplicate prevention still require confirmation.

2. Project statement and rationale

2.1 Project statement

Design and implement a Software Lifecycle Manager using Microsoft Power Platform that centralizes software catalogue management, employee software requests, approval decisions, license assignments and renewal monitoring, while providing distinct employee and IT/procurement experiences.

2.2 Why this project was chosen

- Real workplace relevance: organizations routinely procure SaaS products, developer tools, design tools, analytics tools and collaboration software.

- A complete business lifecycle: the project goes beyond recording a request and follows it through decision, provisioning and renewal.
- Good Power Platform coverage: it gives each major component a meaningful job—Canvas for employee experience, model-driven apps for structured administration, Dataverse for relational data, and Power Automate for workflow.
- Manageable scope: a small catalogue, four core tables and a few focused workflows are sufficient for a convincing prototype.
- Demonstrable business value: the solution can reduce email-based coordination, improve traceability and make recurring license commitments easier to manage.
- Complementary analytics opportunity: a separate Power BI Subscription Creep & Hidden Cost Radar can analyse subscription costs and utilization while SLM manages operational requests and assignments.

2.3 Business problems addressed

Problem	SLM response	Expected benefit
Requests arrive through scattered emails/chats	Central request form and request history	Consistent intake and traceability
Approvals are hard to track	Dataverse approval record and automated approval flow	Clear decision status and comments
IT has no unified administration workspace	Model-driven app with tables, views and forms	Structured record management
License allocations are disconnected from requests	License Assignment table linked to software and request	Traceable provisioning history
Renewal dates can be missed	Scheduled renewal-monitoring flow	Earlier action on upcoming renewals
Recurring software costs are difficult to understand	Complementary Power BI subscription analytics	Visibility into cost and utilization

3. Scope and boundaries

3.1 In-scope capabilities

- Maintain a software catalogue with vendor, category, annual cost, approval requirement and availability.
- Allow employees to browse software and submit a request with department and business justification.
- Allow employees to view their own requests and current status.
- Allow procurement/IT users to view and manage requests, approvals and license assignments.
- Create and update approval records through an automated approval workflow.
- Monitor license renewal dates and send milestone-based notifications.
- Apply appropriate Dataverse permissions and validate role-based access.
- Provide a repeatable end-to-end demonstration using fictional sample data.

3.2 Explicitly out of scope for the capstone

- Connecting to every real software vendor to retrieve live license usage or billing.
- Automatically disabling or revoking production software accounts.
- Processing payments, purchase orders or accounting journal entries.
- Complex multi-level procurement policies, legal reviews or enterprise identity governance.
- Machine-learning-based anomaly detection.

4. Solution architecture and approach

Layer	Component	Responsibility
Experience	Canvas App — Software Request Portal	Employee catalogue, request submission and personal request tracking.
Administration	Model-driven App — Procurement & IT Workspace	Structured management of catalogue, requests, approvals and assignments.
Data	Microsoft Dataverse	Relational records, choice fields, lookups and centralized business data.
Automation	Power Automate	Approval orchestration, notifications and scheduled renewal checks.
Connectors	Dataverse, Approvals, Outlook / Office 365	Read/write business records and communicate decisions/reminders.
Analytics (companion capstone)	Power BI Subscription Creep & Hidden Cost Radar	Analyse recurring costs, license utilization and upcoming renewals.

4.1 Design principles

- Dataverse is the system of record; the apps are different interfaces over the same business data.
- Use lookup relationships rather than duplicating related record names wherever possible.
- Keep employee interactions simple and move administrative record management to the model-driven app.
- Use flows for asynchronous approvals, notifications and scheduled monitoring.
- Separate interface visibility from actual authorization: hiding a button is not security.
- Build the smallest complete lifecycle first, then add polish and optional enhancements.

5. Dataverse data model

Four core Dataverse tables were created as part of the solution.

Table	Purpose	Key columns / fields
Software Catalog	Master catalogue of requestable products.	Software Name (primary name), Vendor, Category (Choice), Annual Cost (Currency), Requires Approval (Yes/No), Is Available (Yes/No), Description.
Software Requests	One record per employee request.	Request Number (primary name), Requester Email, Department (Choice), Software (lookup), Business Justification, ReqStatus (Choice), Submitted On, Decision Date, Decision Comments.
Approvals	Decision record associated with a request.	Approval Name (primary name), Software Request (lookup), Approver Email, Decision (Choice), Comments, Decision Date.
License Assignments	Tracks allocation and renewal of a software license.	Assignment Name (primary name), Software (lookup), Assigned User Email, Software Request (lookup), Start Date, Renewal Date, Annual Cost, Status (Choice).

5.1 Relationships

Parent table	Relationship	Child table	Meaning
Software Catalog	1:N	Software Requests	A software product can be referenced by many requests.
Software Requests	1:N	Approvals	A request can have one or

			more approval records.
Software Catalog	1:N	License Assignments	A product can have many license assignments.
Software Requests	1:N	License Assignments	A request can result in one or more assignments; request reference can be optional for manually created assignments.

In Dataverse, the many side of a one-to-many relationship is implemented through a lookup column on the child table. Creating the relationship manually still results in a lookup-backed relationship.

5.2 Choice fields

Table / column	Options confirmed in project
Software Catalog.Category	Productivity, Development, Design, Analytics, Collaboration, Security
Software Requests.Department	IT, Finance, HR, Marketing, Operations
Software Requests.ReqStatus	Draft, Submitted, Pending Approval, Approved, Rejected, Provisioned
Approvals.Decision	Pending, Approved, Rejected
License Assignments.Status	Available, Assigned, Revoked

Dataverse Choice values are stored as numeric option values. The Submitted option was used in Power Automate with the numeric value 120970001. Other option values should be taken from the actual environment rather than guessed.

6. Canvas App — Software Request Portal

The Canvas App is the employee-facing experience. It is designed for quick discovery and submission rather than exposing employees to the full Dataverse schema.

6.1 Confirmed screens and controls

Screen	Controls	Purpose / status
scrCatalog	lblTitle, txtSearch, galSoftware, btnRequest; My Requests navigation	Catalogue display, search and selected-software navigation. Confirmed as part of the built employee flow.
scrRequest	lblRequestTitle, lblSelectedSoftware, ddDepartment, txtJustification, btnSubmit, btnCancel	Request submission form. Created and submission tested successfully.
scrMyRequests	lblMyRequestsTitle, btnBackCatalog, galMyRequests, ddStatusFilter, btnRefreshRequests	Personal request tracking. User reported this screen was completed.

6.2 Key behaviours implemented

- Selecting a catalogue item stores the selected Dataverse record in varSelectedSoftware and navigates to scrRequest.
- The request screen displays the selected software name.
- Department is populated from the Dataverse Department Choice using Choices('Software Requests'.Department).
- The justification field is multiline.

- The submit action validates required values and uses Patch() to create a Software Requests record.
- The submission stores a generated request number, User().Email, department, software lookup, justification, Submitted status and submission timestamp.
- The successful submission was verified in Dataverse; all fields were reported correct and ReqStatus was Submitted.
- The My Requests gallery filters records by the signed-in user's email and supports status filtering and refresh.

6.3 Important implementation note

The initial submit formula displayed a success notification after Patch(). A production-quality version should capture the Patch result, handle errors with IfError() or Errors(), and navigate only after a confirmed save. This is a recommended hardening task, not a confirmed completed feature.

6.4 Planned Canvas improvements

- Add an Admin Workspace entry point visible only to authorized administrators.
- Add software create/edit form for catalogue maintenance.
- Add an admin request queue with filters and status-management controls.
- Introduce a controlled App Access table or, preferably, enforce permissions through Dataverse security roles and appropriate record access.
- Add clearer loading, success and error states; prevent duplicate clicks/submissions.
- Optionally show request details in a dedicated detail screen.

7. Model-driven App — Procurement & IT Workspace

The model-driven app is the administrative experience. It reuses the same Dataverse tables and is intended for IT/procurement users who need searchable lists, record forms and structured maintenance.

7.1 Confirmed work

- Created a model-driven app named Procurement & IT Workspace inside the Software Lifecycle Manager solution.
- Added the four Dataverse tables: Software Catalog, Software Requests, Approvals and License Assignments.
- Created custom Software Requests views named Pending Requests and Recently Approved.
- Pending Requests is intended to include ReqStatus = Submitted OR ReqStatus = Pending Approval.
- Recently Approved is intended to include ReqStatus = Approved.
- The user confirmed that the views were created and then made available in the model-driven app view selector.

7.2 Remaining model-driven improvements

Area	Recommended work	Acceptance criteria
Software Catalog main form	Group fields into Product information, Commercial details and Availability.	An IT user can quickly inspect and update a product.
Software Requests main form	Create sections for Request details, Requester details, Decision and audit information.	Important information is visible without excessive scrolling.
Approvals main form	Show linked request, approver, decision, comments and decision date.	Approval history is easy to understand.
License Assignments main form	Group software/user, assignment dates, renewal and status.	IT can maintain assignment and renewal details.
Views	Add useful columns and sort order; create Expiring Licenses and Assigned Licenses views.	Users can find operational work quickly.
Business rules	Require decision comments when	Invalid or incomplete records are

	rejected; require renewal date for assigned licenses where appropriate.	discouraged.
Security	Assign appropriate Dataverse roles and test with non-admin access.	Employees cannot gain admin privileges merely by navigating directly to a screen.

8. Power Automate flows

8.1 Flow 1 — SLM – Software Request Approval

Purpose: automate the approval lifecycle when a new software request is submitted.

Element	Configuration / behaviour
Trigger	Microsoft Dataverse — When a row is added, modified or deleted; configured for Added; table Software Requests; organization scope.
Gate condition	ReqStatus equals the Submitted Choice option. The numeric option value used was 120970001.
Create approval row	Add a new row in Approvals, linked to the triggering Software Request, with approver email and Decision = Pending.
Approval action	Start and wait for an approval; Approve/Reject — First to respond; title and details include request context.
Outcome condition	Outcome equals the literal text Approve.
If yes	Update Software Requests to Approved; update the created Approvals row to Approved; record comments and decision date.
If no	Update Software Requests to Rejected; update the created Approvals row to Rejected; record comments and decision date.

Confirmed result: the user reported that the test succeeded. The flow contains two Update a row actions in each outcome branch: one for Software Requests using the trigger Row ID, and one for Approvals using the Row ID returned by Add a new row. The exact final field values should still be checked in Dataverse as part of final acceptance testing.

8.2 Flow 2 — SLM – License Renewal Watchdog

Purpose: monitor license assignments and alert the responsible person before renewal dates, while preventing the same milestone reminder from being sent repeatedly.

Element	Planned / discussed configuration
Trigger	Scheduled cloud flow; recurrence every 1 day.
Retrieve records	Dataverse List rows on License Assignments.
Loop	Apply to each over the List rows value array.
Eligibility	Only process records whose License Assignment Status is Assigned.
30-day reminder	Renewal date is within the 30-day reminder window and 30-Day Reminder Sent = No; send email and set flag to Yes.
7-day reminder	Renewal date is within the 7-day reminder window and 7-Day Reminder Sent = No; send email and set flag to Yes.
Overdue reminder	Renewal date is before today and Overdue Reminder Sent = No; send email and set flag to Yes.

Progress qualification: the scheduled flow, List rows and Apply to each setup were discussed and the build was underway. The conversation does not confirm that all conditions, emails, update actions or a successful end-to-end test were completed. The actual Renewal Date logical name was still requested and had not yet been supplied.

8.3 Flow 2 implementation safeguards

- Retrieve the actual logical/schema name for Renewal Date from Dataverse column properties.
- Use the correct numeric Choice value for Assigned if the connector exposes a numeric status value.
- Prefer date-window checks over exact equality so a missed daily run does not permanently miss a reminder.
- Use consistent date-only comparisons and account for UTC versus local date boundaries.
- After sending an email successfully, update the matching License Assignments row using its unique Row ID and set the relevant reminder flag to Yes.
- Avoid updating a reminder flag before the email succeeds.
- Decide whether overdue reminders should be sent once only or repeated at a defined cadence. The current design is one-time per milestone.

8.4 Flow 3 — License Provisioning (planned)

Purpose: close the loop between approval and actual allocation. When an approved request is marked as ready for provisioning or an IT user completes the allocation action, create a License Assignment linked to the Software Catalog record and Software Request, then update the request to Provisioned and notify the employee.

Step	Planned action
1	Trigger on a deliberate provisioning event, such as a request status changing to an agreed provisioning status or a dedicated IT action.
2	Check that the request is Approved and has not already been provisioned.
3	Create License Assignment with Software lookup, Assigned User Email, Software Request lookup, Start Date, Renewal Date, Annual Cost and Status = Assigned.
4	Update Software Request ReqStatus to Provisioned.
5	Notify requester and/or IT by email.
6	Record enough information to prevent duplicate assignment creation.

This flow is recommended but optional if the capstone deadline is very tight. The trigger should be designed carefully to avoid creating duplicate assignments when the request row is updated.

9. Security, roles and governance

A role-based interface is useful, but it is not a security boundary. A Canvas App Visible property or a variable such as varIsAdmin only controls what the interface displays. Actual authorization must be enforced through Dataverse security roles, table privileges, ownership/team access and appropriate row-level scope.

Persona	Intended access
Employee	Read available Software Catalog records; create requests; read only their own requests and relevant decision information.
Approver / Manager	Read requests assigned or accessible to them; create/update relevant approval decisions.
IT / Procurement Admin	Manage software catalogue, review requests, manage approvals and create/update license assignments.
System administrator	Configure environment, solutions, security roles and application administration.

9.1 Security work remaining

- Decide whether records are user/team-owned or organization-owned and design access accordingly.

- Create or configure Dataverse security roles for Employee, Approver and IT Administrator.
- Grant minimum required table privileges; do not grant broad write access to App Access or all business tables.
- Test with separate test accounts or role assignments, not only the maker/admin account.
- Verify that an employee cannot read another employee's request by changing a gallery filter or directly querying Dataverse.
- Verify that non-admin users cannot create or edit Software Catalog or License Assignment records unless explicitly intended.

10. Testing and acceptance plan

Test ID	Scenario	Expected result
T01	Employee opens catalogue.	Available software products are displayed.
T02	Employee searches by software name.	Gallery filters to matching available products.
T03	Employee submits a complete request.	Dataverse Software Requests row is created with correct lookup, department, email, justification, Submitted status and timestamp.
T04	Employee submits with missing justification.	Validation message appears and no row is created.
T05	Employee opens My Requests.	Only their own requests appear.
T06	New request triggers Flow 1.	Approval record is created and approval request is sent.
T07	Approver approves.	Request and Approval records both become Approved; comments/date are recorded.
T08	Approver rejects.	Request and Approval records both become Rejected; comments/date are recorded.
T09	Flow 2 finds a license due in 30 days.	One 30-day reminder is sent and its flag becomes Yes.
T10	Flow 2 runs again for same assignment/milestone.	No duplicate 30-day reminder is sent.
T11	License becomes due in 7 days or overdue.	Correct milestone reminder is sent once and matching flag is updated.
T12	IT provisions an approved request.	One linked License Assignment is created and request becomes Provisioned.
T13	Employee attempts admin action.	Dataverse security prevents unauthorized write even if the UI is bypassed.
T14	Model-driven views are opened.	Pending Requests and Recently Approved show only matching statuses.

11. Remaining work and priorities

Priority	Work item	Status
P0	Complete Flow 2 date expressions, reminder conditions, emails, flag updates and test.	In progress / not confirmed complete
P0	Verify Flow 1 outcome records and comments in Dataverse for both approval and rejection.	Successful flow test reported; data audit remains
P1	Polish model-driven forms and views for	Remaining

	all four tables.	
P1	Configure and test Dataverse security roles and record-level access.	Remaining
P1	Build Flow 3 license provisioning.	Planned
P1	Harden Canvas submit logic with error handling and duplicate-submit prevention.	Recommended remaining
P2	Add Admin Workspace in Canvas App for catalogue and request management.	Proposed; not confirmed built
P2	Add business rules and form-level validation.	Remaining
P2	Prepare final sample data, demo script, screenshots and presentation.	Remaining

12. Recommended final build sequence

1. Finish and test Flow 2 using a small set of test assignments: one due in 30 days, one due in 7 days, one overdue, one future-dated and one revoked.
2. Audit Flow 1 data in Dataverse for both Approved and Rejected outcomes.
3. Polish model-driven forms, views and navigation; make the IT workspace easy to demonstrate.
4. Implement Dataverse security roles and test with employee and admin personas.
5. Build Flow 3 provisioning if time allows; make duplicate prevention part of the design.
6. Harden Canvas validation and error handling; decide whether the Admin Workspace is necessary given the model-driven admin app.
7. Run the complete acceptance test suite and prepare the final demonstration.

13. Final demonstration storyline

A five-to-seven-minute demonstration can tell one coherent story:

8. Show the employee opening the Canvas App and browsing the Software Catalog.
9. Select a product, explain the business need and submit a request.
10. Show the new Software Requests record in Dataverse or My Requests.
11. Show Flow 1 creating an Approval record and sending the manager an approval request.
12. Approve or reject the request and show both Dataverse records updating.
13. Open the model-driven Procurement & IT Workspace and show the request views and administrative forms.
14. Show an approved request being provisioned into a License Assignment (once Flow 3 is implemented).
15. Show Flow 2 detecting a test license renewal date and sending the appropriate reminder without duplicating it.

14. Risks, assumptions and limitations

Risk / assumption	Mitigation
Dataverse, Approvals and Outlook connector availability depends on training-tenant licensing and permissions.	Validate connectors and environment access before building optional features.
Choice fields may appear as labels or numeric values in Power Automate.	Use actual option values from the environment; do not guess numeric values.
Lookup updates require a related row GUID, not a display name.	Use trigger Row ID or Add a new row output ID as appropriate.
Scheduled flow date comparisons can be affected by time zones.	Use date-only comparisons consistently and test near midnight/time-zone boundaries.
Reminder flags alone do not prevent duplicates if a flow run	Update flags only after successful send; consider run

partially fails.	concurrency and retry behaviour.
UI-only admin checks are bypassable.	Enforce access with Dataverse security roles and row-level privileges.
Sample prices and license data are illustrative.	Label demo data as fictional and avoid presenting it as vendor pricing.

15. Conclusion

Software Lifecycle Manager is a suitable Power Platform capstone because it demonstrates a connected business process rather than isolated controls. The project has a functioning employee request submission path, a Dataverse relational foundation, a model-driven administrative application with custom request views, and a successfully tested approval flow. Completing the renewal monitor, strengthening security, polishing administrative forms and (if time permits) adding provisioning automation will turn the prototype into a coherent end-to-end solution.

The final evaluation should emphasize the design decisions: why the data is relational, why Canvas and model-driven apps serve different personas, why approvals and scheduled reminders belong in Power Automate, and how the workflow improves traceability and control over software procurement and recurring license commitments.

Appendix A — Suggested project vocabulary

Term	Meaning in this project
Software catalogue	List of software products employees may request.
Software request	An employee's request to acquire or access a product.
Approval	A recorded decision associated with a request.
License assignment	A record that a license has been allocated to a user.
Provisioning	The operational step of assigning/access-enabling the approved software.
Renewal monitor	Scheduled process that checks upcoming and overdue renewal dates.
Choice	Dataverse option-set column used for controlled status/category values.
Lookup	Dataverse relationship column that references another table's row.